COSC3106: Theory of Computing

CH#5:

Lec#11: Context Free Grammar

School of Computer Science and Technology

Algoma UNIVERSITY

This lecture

- In this lecture, you will …
  - Understand the Need for CFGs.
  - Define a Context-Free Grammar.
  - Construct CFGs for Languages.
  - Understand Recursive Structure.

- A Context-Free Grammar (CFG) is:

$$G = (V, \Sigma, R, S)$$

Where:

- V: set of variables (non-terminals)
- Σ: set of terminals
- R: set of rules
- S: start variable.

Components of CFG are:

$$G = (V, \Sigma, R, S)$$

- Variables V:

  Uppercase: S, A, B

- Terminals Σ:

  Lowercase: a, b

- Rules R:

  Each line is a production rule

  Form: $A \rightarrow w$

- Start variable:

  S

Algoma UNIVERSITY

- **Ex:**

$$\mathbf{S} \rightarrow \mathbf{AB}$$

$$\boldsymbol{A} \rightarrow \boldsymbol{aA} \mid \boldsymbol{a}$$

$$\boldsymbol{B} \rightarrow \boldsymbol{bB} \mid \boldsymbol{b}$$

- Final string: $\boldsymbol{aabb}$?

✓ Valid strings contains only terminals.

$$\boldsymbol{S} \Rightarrow \boldsymbol{AB}$$

$$\Rightarrow \boldsymbol{aAB}$$

$$\Rightarrow \boldsymbol{aaB}$$

$$\Rightarrow \boldsymbol{aabB}$$

$$\Rightarrow \boldsymbol{aabb}$$

Language of a CFG:

$$L(G) = \{w \in \Sigma^* \mid S \overset{*}{\Rightarrow} w\}$$

- The language of the Grammar includes all strings that:
  - Start from S
  - End with only terminals
  - After finite number of steps

- **One-Step Derivation:**

Ex: If $A \rightarrow w$

$$\Rightarrow uAv = uwv$$

- Replace one variable using a rule.

- **Multiple Steps Derivation:**

If $S \rightarrow AB$

$A \rightarrow aA \mid a$

$B \rightarrow bB \mid b$

$$S \overset{*}{\Rightarrow} aabb$$

- Means:
  - Zero or more steps
  - Finite number of rule applications

- **Ex:**

$$L = \{a^n b^n \mid n \geq 0\}$$

  - #a's = #b's
  - Not regular
  - Context-free

- **Ex: Define a CFG for $L = \{a^n b^n \mid n \geq 0\}$?**

$$S \rightarrow \epsilon \mid aSb$$

  - How it works ?

Start from S:

$$S \Rightarrow aSb$$

$$\Rightarrow aaSbb$$

$$\Rightarrow aabb$$

Algoma UNIVERSITY

- **Ex**: Given a CFG :

$$S \rightarrow \epsilon \mid aSb$$

  - Define the language L(G)

Start from S:

$$S \Rightarrow aSb$$

$$\Rightarrow aaSbb$$

$$\Rightarrow aaaaa \ldots S \ldots bbbbb$$

$$\Rightarrow a^n S b^n$$

Then substitute $S \rightarrow \epsilon$

$$\Rightarrow L(G) = a^n b^n$$

Algoma UNIVERSITY

- CFGs are powerful because they can model recursion and matching (dependencies).

  - Regular languages:
    - Can only "read" symbols
    - No memory of previous structure

  - CFGs:
    - Can remember structure using recursion
    - Can match pairs (like open/close)
      - Think of a as "(" and b as ")" :

Ex: $aabb \rightarrow (())$; This is proper nesting.

- CFGs are powerful because they can model recursion and matching (dependencies).
  - S calls itself inside the rule.
  - This means we can repeat the same pattern again and again.
  - This is recursion, and it allows us to build nested structures like parentheses.
  - Ex: $abaaabbb$

$$\Rightarrow ()((()))$$

- CFGs describe languages where elements depend on each other in a structured, recursive way.